

Image Search — Manual Test Plan

Local Multimodal File Search. Fill in **Result** (pass / fail / partial) and **Remark** as you go. Starred rows are the ones that matter most if you are short of time.

Configuration these tests assume	
Image model	OpenCLIP ViT-B/32, 512-dim
Image relevance floor	0.25
Image tail trim	0.90 of the best image hit
Text relevance floor	0.55 (bge, 1024-dim)
Text tail trim	0.95 of the best text hit
Confident-match line	0.62
Visual indexing covers	standalone image files only
OCR covers	page + embedded + standalone images
Captioning	off (~390s per image on CPU)
Results per page	10

Scores are shown for reference only — the interface describes match strength in words. A CLIP score near 0.30 is a good visual match; a bge score near 0.70 is a good text match. The two scales are not comparable.

A. Setup and sanity

Confirm the machine is in the configuration the rest of the plan assumes.

#	What to do	Expected	Result	Remark
A1	Open the Models tab.	Device cpu , OCR engine tesseract . A note says image search is on and captioning is off.		
A2 ★	Check the index status shown in the Search tab sidebar.	Two collections: text chunks and images. Image count is non-zero (794 at the time of writing).		
A3	Search anything with the filter set to Images .	Results return within a few seconds. The first query after a restart is slower (the model loads).		
A4	Disconnect the network entirely, then repeat A3.	Identical results. Nothing in the app requires internet after install.		

B. Getting images in

Formats, duplicates, and the guarantee that your originals are never touched.

#	What to do	Expected	Result	Remark
B1	Index a folder holding .jpg, .jpeg, .png files.	All are accepted. The count reported before indexing matches what appears in the Library.		
B2 ★	Note the file size and modified date of one original, then index its folder and check the original again.	Byte-identical, same timestamp, still in place. The app copies , never moves.		
B3	Index the same folder a second time.	It reports the files as already known. No duplicates appear in the Library.		
B4	Take one image, copy it under a different filename, index both.	Treated as one file: identity is the content hash, not the name.		
B5	Index a folder containing a .webp, .bmp, .tif and a .heic.	Supported types ingest; unsupported ones are skipped or marked failed with a readable message. Nothing crashes the batch.		
B6	Index a folder with images plus a .exe and a .zip.	Non-image files are skipped silently. The image count is unaffected.		
B7	Rename an image to .png when it is really a .jpg, then index it.	Detected by content, not extension. It ingests correctly.		
B8	Index a deliberately corrupt/truncated image file.	That one file is marked failed with a message. Every other file in the batch still completes.		
B9	Index a folder of 100+ images and watch the progress bar.	Progress advances; the UI stays responsive; the count at the end matches.		
B10	While a large batch is indexing, run a search.	Text results return immediately. Image results may be skipped with a note while the models are busy -- that is by design, not a failure.		

C. Reading text out of images (OCR)

OCR is governed by neither image switch and always runs.

#	What to do	Expected	Result	Remark
C1 ★	Index a screenshot containing readable text. Search a distinctive phrase from it.	The screenshot is found. The match is labelled as coming from OCR.		
C2	Search an exact number that appears in a screenshot (an invoice total, a reference).	Found and labelled exact match , not a similarity score.		
C3	Index a photo of a page of printed text and search a phrase from it.	Found via OCR. Quality depends on focus and lighting.		
C4	Index a photo of handwriting and search a word from it.	Likely not found. Tesseract is trained on printed text; this is an expected limitation, not a bug.		
C5	Index a rotated (90 degrees) screenshot of text and search a phrase.	May fail. Note the outcome -- OCR orientation handling is worth knowing about.		
C6	Index an image whose only text is a large logo/wordmark.	The wordmark may be OCR'd. A single repeated word may be filtered as low-information and not indexed.		
C7	Index a photo with no text at all and search words unrelated to it.	No OCR text is stored. The file is still searchable by appearance.		

D. Searching by appearance -- specific things

The core capability: describing a picture you cannot name.

#	What to do	Expected	Result	Remark
D1 ★	Search for a concrete object you know is in one photo (e.g. a red car).	That photo appears at or near the top.		
D2	Search the same thing with a wrong colour (a blue car).	Ranking changes. The red car should rank lower or drop out.		
D3	Search an animal you have exactly one photo of (a dog).	That photo is returned, typically as the only result.		
D4	Search a full description: subject, clothing, colour (man in a yellow shirt).	Strong, precise match. Detailed descriptions are what CLIP handles best.		
D5	Search a scene rather than an object (mountains at sunset).	Relevant landscapes returned if any exist; nothing if not.		
D6	Search an action (a person riding a bicycle).	Matching photos returned. Actions work less reliably than objects.		
D7	Search using a word that never appears in any filename.	Still finds the image. This is the whole point -- filename search cannot do it.		
D8	Misspell a query slightly (a red mercedes).	Still finds it. Meaning survives small spelling errors.		

E. Searching by category -- the generic-word case

Bare nouns used to return nothing. Query templating fixed that; verify it holds.

#	What to do	Expected	Result	Remark
E1 ★	Search person.	Many photos of people returned (25 on the reference library), not zero and not one.		
E2 ★	Search human.	Similar set to E1. Before the fix this returned nothing at all .		

#	What to do	Expected	Result	Remark
E3	Search people.	Photos containing more than one person rank highest.		
E4	Search vehicle, then car.	Both return vehicles. car is usually stronger -- CLIP handles concrete words better than abstract categories.		
E5	Compare the results of E1 against scrolling the Library by eye.	Recall is much improved but not exhaustive . This is ranked retrieval, not classification: a person tiny or obscured in frame can still fall below the floor.		
E6	Search animal, then the specific species.	The specific word scores higher. Note the difference; it is the same effect as E4.		
E7	Search a one-word colour (yellow).	Images dominated by that colour rank up. Weak signal on its own -- expected.		
E8	Search a plural vs singular (dogs vs dog).	Similar but not identical result sets. Worth noting the difference.		

F. What gets indexed, and what does not

Only standalone image files get a visual vector. Pictures inside documents do not.

#	What to do	Expected	Result	Remark
F1 ★	Index a PDF containing photographs. Search a description of one of those photographs.	Not found by appearance. Images inside documents are deliberately excluded from visual search -- OCR already reaches them, and one slide deck alone produced 2054 of them.		
F2 ★	Search text that appears inside a picture embedded in that same PDF.	Found , via OCR. Narrowing visual indexing did not narrow OCR.		
F3	Save that same photograph out of the PDF as a .jpg and index it on its own. Repeat the F1 search.	Now found by appearance. Standalone image files are the indexable kind.		
F4	Index a DOCX containing images and check the Library entry.	One file, its embedded images visible in the details. No visual vectors for them.		
F5	Index a scanned PDF (pages are images, no text layer) and search a phrase from a page.	Found via OCR of the page image. Scanned documents stay fully searchable.		
F6	Filter to Documents and search a visual description.	No image-file results. The filter applies to the file a match came from.		
F7	Filter to Images and search a phrase that only exists in a PDF.	No results. Confirms the filter is applied in the store, not after retrieval.		

G. Ranking, filtering and presentation

How results are ordered, trimmed and shown.

#	What to do	Expected	Result	Remark
G1	Run a query matching more than ten images.	Ten shown per page with working page navigation. Page 2 shows different files.		

#	What to do	Expected	Result	Remark
G2 ★	Search something the library certainly does not contain (a submarine in a desert).	No results, or a clear warning that the best match is weak. It must not present noise as an answer.		
G3	Search deliberate gibberish (asdkjfh qwoieur).	Ideally nothing. A few weak results is a known limitation -- CLIP has no clean noise threshold. Note what you get.		
G4	Search a term matching both a document and a photo, filter Everything .	Both kinds appear. Text results must not crowd out image results, and vice versa.		
G5	Check the match strength wording on a strong result vs a weak one.	Described in words (strong / good / weak), never as a bare fraction.		
G6	Find a result that matched both by caption and by appearance.	Shown as one file, not two rows. Passages are grouped per file.		
G7	Change the filter from Everything to Images with a query already typed.	Results refresh and paging resets to page 1.		
G8	Click through to open an image result.	The correct file opens. The thumbnail shown matches the file opened.		
G9	Search an empty string / only spaces.	No error. Nothing is searched.		
G10	Search a very long sentence (30+ words).	Returns something sensible or nothing. No crash, no timeout.		

H. Captions (currently switched off)

Captioning costs about 390s per image on this CPU and is disabled. Verify that state.

#	What to do	Expected	Result	Remark
H1	Index a new photo and open its details in the Library.	No caption text. The photo is still searchable by appearance and by OCR.		
H2 ★	Search a description of one of the five photos captioned earlier (the red Mercedes, the train, the trophy).	Found, and the caption text is shown as the snippet. Old captions survive; new images simply do not get one.		
H3	Confirm the Models tab explains why captioning is off.	A note gives the reason and the cost, not just a silent absence.		
H4	(Optional, slow) Set FS_ENABLE_CAPTIONING=true, restart, add one photo, wait.	A caption appears after roughly 6-7 minutes. Image search is unavailable while captioning runs -- expected in low-memory mode.		

I. Failure, recovery and durability

What happens when something goes wrong. Do these last.

#	What to do	Expected	Result	Remark
I1	Search while a large batch is still indexing.	Text results immediate. Image results may be skipped with a note rather than making you wait.		
I2	Stop the backend with Ctrl+C, restart it, search again.	Everything indexed before the stop is still searchable.		
I3 ★	Index some new images, then force-kill the backend from Task Manager before it is stopped cleanly. Restart and search for those images.	Known bug: the vectors are lost while the database still records them as indexed, so those images silently never appear. Confirm, then avoid force-killing.		

#	What to do	Expected	Result	Remark
I4	Delete an image from the Library, then search for its content.	Gone from results. Its vectors are removed too, not just the row.		
I5	Delete the underlying original from the app's store, then search.	The result may still list but opening it fails gracefully with a message.		
I6	Restart the backend and watch the startup log.	Nothing is re-queued when the library is fully indexed. It should not re-index on every start.		

J. Edge cases

Unusual inputs. Failures here are worth recording even if minor.

#	What to do	Expected	Result	Remark
J1	Index a very small image (under 100x100).	Accepted as a standalone file. Note whether it is findable by appearance.		
J2	Index a very large image (8000x6000 or larger).	Ingests without exhausting memory. May be slow. It must not kill the backend.		
J3	Index an image with transparency (PNG with alpha).	Handled without artefacts. The thumbnail looks correct.		
J4	Index a CMYK JPEG.	Converted correctly. Colours in the thumbnail are not inverted or washed out.		
J5	Index an animated GIF.	Accepted or cleanly rejected. Not a crash.		
J6	Index a photo with EXIF rotation set.	The thumbnail is the right way up, matching how it looks in Photos.		
J7	Index a completely black and a completely white image, then search a dog.	Neither should rank highly. Featureless images must not become universal matches.		
J8	Index two near-identical photos (burst shots) and search their content.	Both returned as separate files, ranked adjacently.		
J9	Index an image with a very long filename and unicode characters.	Ingests and displays without breaking the layout.		
J10	Index an image inside a folder path containing spaces and accents.	Path handled correctly.		
J11	Search a query in a language other than English (un chien).	May work -- the model has some multilingual signal. Record what you observe.		
J12	Search using an emoji only.	No crash. Result quality is unimportant.		